

Interface with Default & Static

- Default បង្កើតនៅក្នុង Interface មានន័យថាបើ Client អត់ override Method ទេវានឹងចាប់យកពី default from Interface.
- Static : ពេលយើងហៅវាមកប្រើនៅក្នុង Client we need to get it from original class or Interface. Ex `System.out.println("static method is : " + Multiple.isEmpty(integerList));`
- We can not have "default" in the class. This default keyword use to create the Method inside the Interface only.

1-Interface

```
package com.learn.java.defaults;

import java.util.List;

public interface Multiple {

    int Multiplier(List<Integer> integerList);

    default int size(List<Integer> integerList) {
        System.out.println("Inside Multiplier Interface");
        return integerList.size();
    }

    static boolean isEmpty(List<Integer> integerList) {

        return integerList!=null && integerList.size()>0;
    }

}
```

2-Implementation

```
Multiple.java MultiplierImpl.java MultiplierClient.java
Java-8-to-Java-25 src/main/java com.learn.java.defaults MultiplierImpl size(List<Integer>): int
1 package com.learn.java.defaults;
2
3 import java.util.List;
4
5 public class MultiplierImpl implements Multiple{
6
7
8     @Override
9     public int Multiplier(List<Integer> integerList) {
10
11         return integerList.stream()
12             .reduce(1,(x,y)-> x*y);
13
14     }
15
16     public int size(List<Integer> integerList) {
17         System.out.println("Inside MultiplierImpl class");
18         return integerList.size();
19     }
20
21 }
22
```

3-Client

```
Java-8-to-Java-25 > src/main/java > com.learn.java.defaults > MultiplierClient >
1 package com.learn.java.defaults;
2
3 import java.util.Arrays;
4
5
6 public class MultiplierClient {
7
8     public static void main(String[] args) {
9
10         // create object from Interface then have to implemnt one of that
11         Multiple multiplier = new MultiplierImpl();
12
13         List<Integer> intergerList = Arrays.asList(1,3,5);
14
15         System.out.println("Result is : " + multiplier.Multiplier(intergerList));
16         System.out.println("default method is : " + multiplier.size(intergerList));
17         System.out.println("static method is : " + Multiple.isEmpty(intergerList));
18
19         // we can use the static method by called the Class or Interface
20     }
21
22 }
23
```

Multiple Inheritance

- Extends: ប្រើជាមួយ Class Or Interface វាអាចធ្វើការ inheritance

បានតែមួយទេ

Example:

```
Java
// Class inheritance using extends
class Animal {
    void eat() {
        System.out.println("Animal eats");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}
```

Java



```
// Interface inheritance using extends
interface Animal {
    void eat();
}

interface Mammal extends Animal {
    void giveBirth();
}
```

- Implements ប្រើតែជាមួយ Class ទេ គេហៅវាថា class implements and interface to provide concrete implementation.
- Class មួយនឹងអាចធ្វើការ implements បានច្រើនពី multiple interface

Java



```
interface Animal {
    void eat();
}

interface Mammal {
    void giveBirth();
}

class Dog implements Animal, Mammal {
    @Override
    public void eat() {
        System.out.println("Dog eats");
    }

    @Override
    public void giveBirth() {
        System.out.println("Dog gives birth");
    }
}
```

Override Method

យើងធ្វើការ ប្រើប្រាស់ Method មានឈ្មោះដូចទៅនឹង Parent មានន័យថាពេលដែលយើងធ្វើការ Implementation from parent រាល់ Method in Parent យើងអាចប្រើប្រាស់បាន តែបើយើង Override Method from Parent method វានឹងយក Method from child class មកប្រើ

ពេលដែល class implement from Interface that have the same name in both Interface 1 and Interface 2 so on. So we need to Override the Method in the Children class